

DemandTest: Demand-Driven Static Discovery of Project-Specific Knowledge for Efficient Intention-Aligned Unit Test Generation

Yunqi Li Jinjie Guo

University of Illinois Urbana-Champaign · yunqili4@illinois.edu · jinjie2@illinois.edu · CS527 project summary, September 2026

Target problem and motivation

Given a focal method, a large Java codebase, and a natural-language *validation intention*, the task is to generate a unit test that compiles, runs, and validates that intention, with a locally hosted model and at a fraction of an agent's cost. Industrial tests validate requirements, not coverage [1], and writing one needs project-specific knowledge: which factory builds a valid input, which fixtures and mocks the project uses, what is observable for the assertion. Repository agents such as OpenHands [2] find that knowledge by reading files one at a time. Context-injection generators such as KTester [3] and CATGen [4] inject fixed patterns with no criterion for when enough has been collected. A 2026 review of 116 studies [5] lists two project-level systems and no efficiency metric within its window. The setting is ICSE 2027 Industry Challenge 8 [6], whose requirements are intention alignment, scalability, at least 50% fewer tokens and 20% less time than an agent at comparable quality, private-model deployability, and evaluation on compile rate, pass rate, alignment, cost, time, and files inspected.

Plan for approach design

Thesis: the knowledge a test needs is largely determined by types and the intention, so compute it before any model call. **S0** indexes the project once with JavaParser [7]: constructors, factories, builders, singletons, observables, fields read, and every existing test's fixtures and callees. **S1** derives a typed *demand set*: receiver, one need per argument bound to its precondition sentence, setup needs, oracle needs from the expected results under one cue rule with negation handling, and the test idiom; what it cannot bind is recorded as semantic-gap flags. **S2** resolves each need to the cheapest accessible construction recipe, expands only along unresolved needs, and stops when a *structural* context-sufficiency predicate Σ holds; otherwise it falls back to the two existing tests with the largest demand overlap, a static analog of TestTailor's path-proximal tests [8], each shown with its demand diff. **S3** renders one packet, with trigger hints for oracles in the spirit of exLong [9], and makes one call to an open-weight model. **S4** repairs statically, compiles, and runs. **S5** makes at most one repair call. Σ is claimed only for evidence that survives rendering, so every run carries a status: *sufficient*, *sufficient-with-gaps*, *budget-limited*, or *fallback*. The research question is whether typed obligations decide *which* context to retrieve and *when to stop* better than similarity ranking or fixed expansion at the same allowance; static analysis, compact context, and few calls are shared with prior work [3, 4, 10] and are not claimed.

Plan for evaluation design

Subjects: the 13 Java projects of IntentionTest's benchmark [1], with intentions reverse-engineered from existing tests as they did, 10% human-checked, plus 30 intentions written independently from the focal method only. No information derived from the held-out test enters any system; the agent's checkout loses the method and its git history; results are split by whether a close clone test is visible. Primary metric: *aligned success rate* over all tasks, requiring compile, pass, dynamic focal execution via JaCoCo, and the top judge score, with the judge validated against blinded, adjudicated human ratings, since LLM-written oracles tend to encode actual rather than intended behaviour [11]. Efficiency: tokens, first-use and amortized time, online files, in four accounting buckets shared by every system. Baselines: OpenHands at 10, 30, and 60 iterations, a simplified IntentionTest reimplementation, and a compact static-context baseline sharing the generator. Two controlled experiments: *selection*, varying only the selector over one candidate pool at fixed allowances; and *stopping*, varying only the stopping policy over nested packets. Mutation score with PIT [12] on a subset. Non-inferiority is predefined at five points with sensitivity at three and two. A 150-task pilot across three contrasting projects decides *proceed*, *redesign*, or *inconclusive* before the full evaluation.

Expected outcome

A Pareto plot of aligned success against tokens with confidence intervals, one point per configuration and agent budget, and a per-system table with index and online costs separated. Expected: comparable aligned success below half the agent's tokens and 80% of its time, with the largest gap on online files inspected. Expected and reported failure modes: Σ holds while the call fails, attributed by manual analysis to premature stops or generation errors; intentions with relational or state preconditions the static analysis cannot bind; large repositories that do not build. Deliverables: the generator, a reusable ledger harness any generator can write to, the reproduced benchmark, and a poster. Not claimed: bug finding, fine-tuning, industrial evaluation, which is proxied and stated as a limitation.

References

1. B. Qi et al. Generating project-specific test cases with requirement validation intention. arXiv:2507.20619, 2025.
2. X. Wang et al. OpenHands: An open platform for AI software developers as generalist agents. ICLR, 2025.
3. A. Li et al. Knowledge matters: Injecting project and testing knowledge into LLM-based unit test generation. 2026.
4. J. Chen et al. Context matters: Improving the practical reliability of LLM-based unit test generation. arXiv:2607.19682, 2026.
5. J. Zhang, X. Hu, C. Gao, X. Xia, S. Li. Enhancing automated unit test generation with large language models: A systematic literature review. ACM TOSEM 35(9):260, 2026.
6. ICSE 2027 Industry Challenge 8: Efficient project-specific test generation with validation intentions. conf.researchr.org/track/icse-2027/icse-2027-industry-challenge, 2026.
7. JavaParser. javaparser.org.
8. X. Zhou, Y. Lou, J. Dong, D. Hao. TestTailor: Generating high-coverage tests via path-proximal tests with LLMs. Proc. ACM Softw. Eng. 3(FSE):277–298, 2026.
9. J. Zhang, Y. Liu, P. Nie, J. J. Li, M. Gligoric. exLong: Generating exceptional behavior tests with large language models. ICSE, 2025.
10. J. Ferrer, F. Chicano. Optimizing context and cost in LLM-based unit test generation: A study on external dependency retrieval strategies. Expert Systems 43(10), 2026.
11. M. Konstantinou, R. Degiovanni, M. Papadakis. Do LLMs generate test oracles that capture the actual or the expected program behaviour? arXiv:2410.21136, 2024.
12. H. Coles et al. PIT: A practical mutation testing tool for Java. ISSTA, 2016.